

File Map & Architecture Documentation

Orders4U Application

Authors	Owen Lindsey & Brennan Bania
Course	CST-323
Instructor	Professor Sluiter
Date	23 January 2026

Document Purpose

This document serves as the **contract between Student 1 (Backend) and Student 2 (Frontend)**. It maps backend routes to frontend templates and provides descriptions of each file in the project.

 **Important**

If a page does not appear in this file-map, it should not be implemented.

Table of Contents

- [Route to Template Mapping](#)
- [Template Summary](#)
- [File Descriptions](#)
- [Security Notes](#)
- [Technology Stack](#)
- [Application Screenshots](#)

Route to Template Mapping

Authentication Routes (Public)

HTTP Method	Route	Template	Description
GET	/users/login	login.html	Display login form
POST	/login	(redirect)	Process login credentials (Spring Security)
GET	/users/register	register.html	Display registration form
POST	/users/processRegister	register.html	Process new user registration
POST	/logout	(redirect)	End user session, redirect to login

Home Route (Authenticated)

HTTP Method	Route	Template	Description
GET	/	home.html	Display home page with Inventory & Admin links

Admin Routes (ROLE_ADMIN Required)

 **Note**

All /admin/* routes require `ROLE_ADMIN` authentication.

HTTP Method	Route	Template	Description
GET	/admin	admin.html	Display list of all users
GET	/admin/editUser/{id}	editUser.html	Display edit form for user
POST	/admin/updateUser	(redirect)	Process user updates
GET	/admin/confirmDelete/{id}	confirmDelete.html	Display delete confirmation
GET	/admin/deleteUser/{id}	(redirect)	Process user deletion
GET	/admin/toggleUser/{id}	(redirect)	Toggle user enabled/disabled status

Order Routes (Authenticated)

 **Note**

All /orders/* routes require authentication.

HTTP Method	Route	Template	Description
GET	/orders	allOrders.html	Display list of all orders
GET	/orders/showOrder/{id}	showOrder.html	Display single order details
GET	/orders/editOrder/{id}	editOrder.html	Display edit form for order
POST	/orders/processEditOrder	(redirect)	Process order updates
GET	/orders/newOrder	newOrder.html	Display new order form
POST	/orders/processNewOrder	(redirect)	Process new order creation
GET	/orders/deleteOrder/{id}	(redirect)	Delete an order

Template Summary

Authentication Templates

Template	Purpose
login.html	User authentication form
register.html	New user account creation form

Admin Templates

Template	Purpose
admin.html	Admin panel displaying all users
editUser.html	Admin form for editing user details
confirmDelete.html	Admin confirmation dialog for user deletion

Application Templates

Template	Purpose
home.html	Landing page for authenticated users
allOrders.html	List view of all orders
showOrder.html	Detail view of a single order
editOrder.html	Form for editing an order
newOrder.html	Form for creating a new order

Navigation Bar Behavior

User State	Navigation Items
Not logged in	Login, Register
Logged in (any role)	Logout
Logged in (ADMIN)	User Admin, Logout

File Descriptions

Application Entry Point

`App.java`

Location: `com.gcu.App`

The main entry point for the Spring Boot application. This class bootstraps the entire application using Spring Boot's auto-configuration mechanism. Upon execution, it initializes the Spring ApplicationContext, triggers component scanning for all `@Controller`, `@Service`, and `@Repository` beans, and starts the embedded Tomcat server to begin accepting HTTP requests.

Configuration

`SecurityConfig.java`

Location: `com.gcu.config.SecurityConfig`

Configures Spring Security for the application, defining authentication and authorization rules. This class defines the `SecurityFilterChain` bean for HTTP security configuration, establishing URL-based authorization rules that grant public access to registration and login routes (`/users/register`, `/users/processRegister`, `/users/login`), restrict admin routes (`/admin/**`) to users with `ROLE_ADMIN`, and require authentication for all other routes.

The configuration specifies a custom login page at `/users/login` with form submissions posting to `/login` (Spring Security default). Logout behavior redirects users to the login page. The class also provides a `PasswordEncoder` bean using the BCrypt algorithm for secure password hashing.

Controllers

HomeController.java

Location: com.gcu.controllers.HomeController

Handles requests for the application's home page.

Endpoints:

Method	Route	Returns
GET	/	home

UserController.java

Location: com.gcu.controllers.UserController

Manages user authentication and registration functionality. This controller depends on `UsersRepository` for database access and `PasswordEncoder` for BCrypt password hashing.

Endpoints:

Method	Route	Returns
GET	/users/login	login
GET	/users/register	register (with empty UserModel)
POST	/users/processRegister	register (errors) or redirect to /users/login

The controller validates that password and confirmation fields match, checks for duplicate usernames in the database, encrypts passwords using BCrypt before storage, and assigns the default `ROLE_USER` to all new registrations.

AdminController.java

Location: com.gcu.controllers.AdminController

Provides administrative functionality for user management. All routes in this controller require `ROLE_ADMIN` authentication. The controller depends on `UsersRepository` for database access and `PasswordEncoder` for BCrypt password hashing.

Endpoints:

Method	Route	Returns
GET	/admin	admin (with all users)
GET	/admin/editUser/{id}	editUser (with user data)
POST	/admin/updateUser	redirect to /admin
GET	/admin/confirmDelete/{id}	confirmDelete (with user)
GET	/admin/deleteUser/{id}	redirect to /admin
GET	/admin/toggleUser/{id}	redirect to /admin

Safety mechanisms prevent administrators from demoting or deleting their own accounts. Password updates only occur when a new value is explicitly provided. The toggle endpoint allows enabling or disabling user accounts without deletion.

OrdersController.java

Location: com.gcu.controllers.OrdersController

Handles all CRUD operations for order management. This controller depends on `OrdersDataService` for business logic layer operations.

Endpoints:

Method	Route	Returns
GET	/orders	allOrders (with all orders)
GET	/orders/showOrder/{id}	showOrder (with order data)
GET	/orders/editOrder/{id}	editOrder (with order data)
POST	/orders/processEditOrder	redirect to /orders
GET	/orders/newOrder	newOrder (with empty model)
POST	/orders/processNewOrder	redirect to /orders
GET	/orders/deleteOrder/{id}	redirect to /orders

Data Layer

DataAccessInterface.java

Location: com.gcu.data.DataAccessInterface<T>

A generic interface defining the contract for data access operations. Implements the Data Access Object (DAO) pattern.

Methods:

Method	Description
getById(int)	Retrieve a single item by ID
getAll()	Retrieve all items
create(T)	Create a new item
update(T)	Update an existing item
deleteById(int)	Delete an item by ID

OrdersDataService.java

Location: com.gcu.data.OrdersDataService

Service layer implementation for order data operations. Implements `DataAccessInterface<OrderModel>`. This service depends on `OrdersRepository` for Spring Data JDBC operations, `DataSource` for potential raw queries, and `Mapper` for entity/model conversion.

The service converts between `OrderEntity` (persistence layer) and `OrderModel` (business logic layer), provides CRUD operations through the repository, and acts as an abstraction layer between controllers and the database.

OrdersRepository.java

Location: com.gcu.data.OrdersRepository

Spring Data JDBC repository interface for order persistence. Extends `CrudRepository<OrderEntity, Integer>`, inheriting standard methods including `save()`, `findById()`, `findAll()`, `deleteById()`, and `count()`.

UsersRepository.java

Location: com.gcu.data.UsersRepository

Spring Data JDBC repository interface for user persistence. Extends `CrudRepository<UserEntity, Integer>`.

Custom Methods:

Method	Description
findByUsername(String)	Find user by unique username

UsersDetailsService.java

Location: com.gcu.data.UsersDetailsService

Implements Spring Security's `UserDetailsService` interface for authentication. This service depends on `UsersRepository` for database access.

When Spring Security requires user authentication, this service loads user data by username, maps the `UserEntity` to Spring Security's `UserDetails` object, converts user roles to `GrantedAuthority` objects, and respects the `enabled` flag for account activation control.

Models

UserEntity.java

Location: com.gcu.models.UserEntity

JPA entity class mapped to the `USERS` database table.

Fields:

Field	Type	Description
id	int	Primary key (auto-increment)
username	String	Unique username
password	String	BCrypt-encrypted password
role	String	User role (<code>ROLE_USER</code> , <code>ROLE_ADMIN</code>)
enabled	boolean	Account activation status

UserModel.java

Location: com.gcu.models.UserModel

Data Transfer Object (DTO) for user-related form submissions.

Fields:

Field	Type	Description
id	int	User identifier
username	String	Username input
password	String	Password input
confirmPassword	String	Password confirmation

OrderEntity.java

Location: com.gcu.models.OrderEntity

JPA entity class mapped to the `ORDERS` database table.

Fields:

Field	Type	Column	Description
id	int	ID	Primary key
order_number	String	ORDER_NUMBER	Order identifier
product_name	String	PRODUCT_NAME	Product name
price	int	PRICE	Unit price
quantity	int	QTY	Quantity ordered

OrderModel.java

Location: com.gcu.models.OrderModel

Data Transfer Object (DTO) for order-related form submissions and business logic.

Fields:

Field	Type	Description
id	int	Order identifier

Field	Type	Description
order_number	String	Order reference number
product_name	String	Product name
price	int	Unit price
quantity	int	Quantity ordered

Mapper.java

Location: com.gcu.models.Mapper

Utility class for converting between entity and model objects.

Methods:

Method	Description
toEntity(OrderModel)	Convert OrderModel to OrderEntity
toModel(OrderEntity)	Convert OrderEntity to OrderModel

Resources

application.properties

Location: src/main/resources/application.properties

Spring Boot configuration file containing MySQL/MariaDB database connection settings. The configuration enables automatic database creation with the createDatabaseIfNotExist=true parameter and sets schema initialization mode to always to ensure the database schema is created on each application startup.

schema.sql

Location: src/main/resources/schema.sql

Database schema initialization script executed on application startup. This script creates the **ORDERS** table for storing order information and the **USERS** table for storing user credentials and roles.

Security Notes

 Warning

Security Considerations

All admin routes (`/admin/**`) require `ROLE_ADMIN` authentication, ensuring only authorized administrators can access user management functionality. POST routes process form submissions and may redirect on success or re-render with error messages on the same template that initiated the action.

Passwords are encrypted using BCrypt before storage, providing industry-standard security for credential management. As a safety mechanism, administrators cannot demote or delete their own accounts, preventing accidental logout. User accounts can be disabled without deletion, allowing administrators to temporarily restrict access while preserving account data.

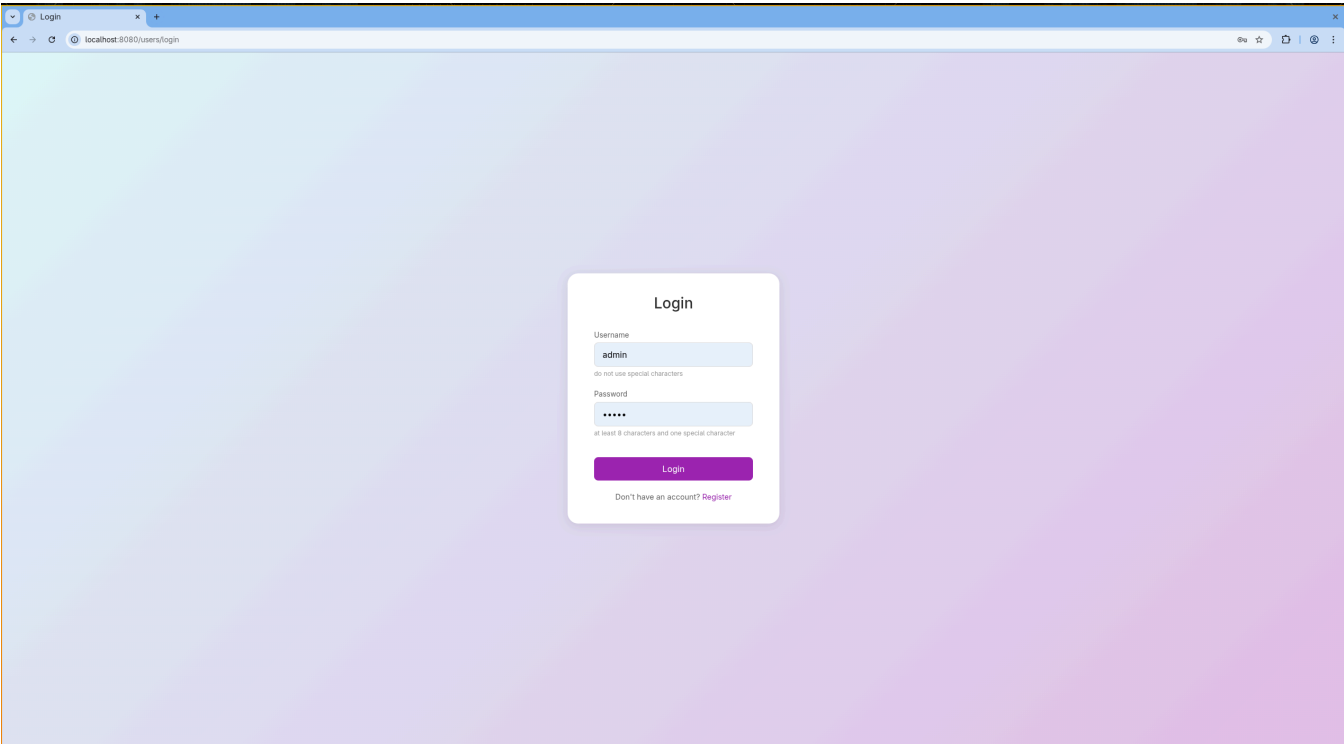
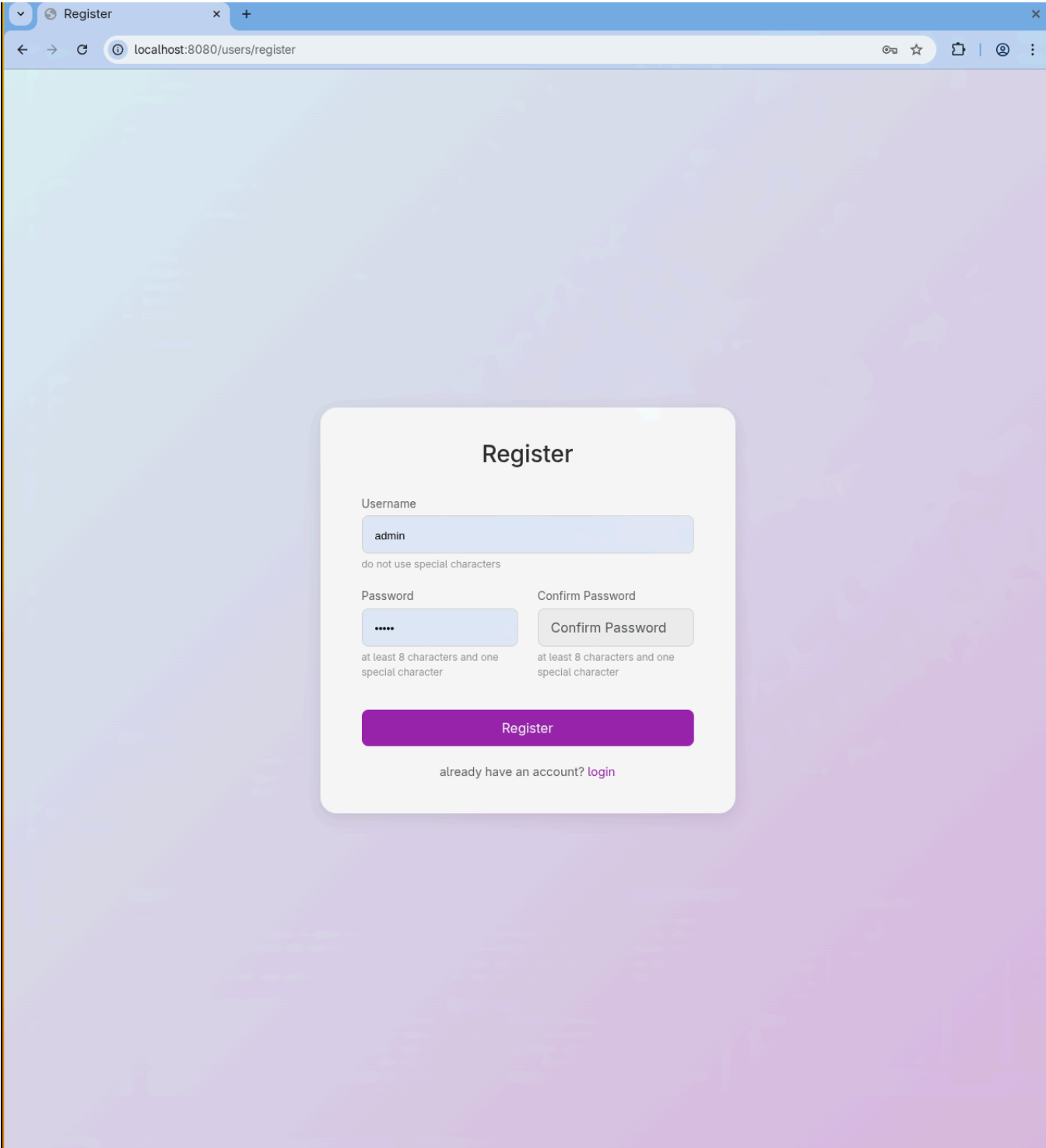
Technology Stack

Component	Technology
Framework	Spring Boot 3.2.0
Template Engine	Thymeleaf
Security	Spring Security 6
Database Access	Spring Data JDBC
Database	MySQL/MariaDB
Build Tool	Maven
Java Version	17

Application Screenshots

The following pages are implemented in the Orders4U application:

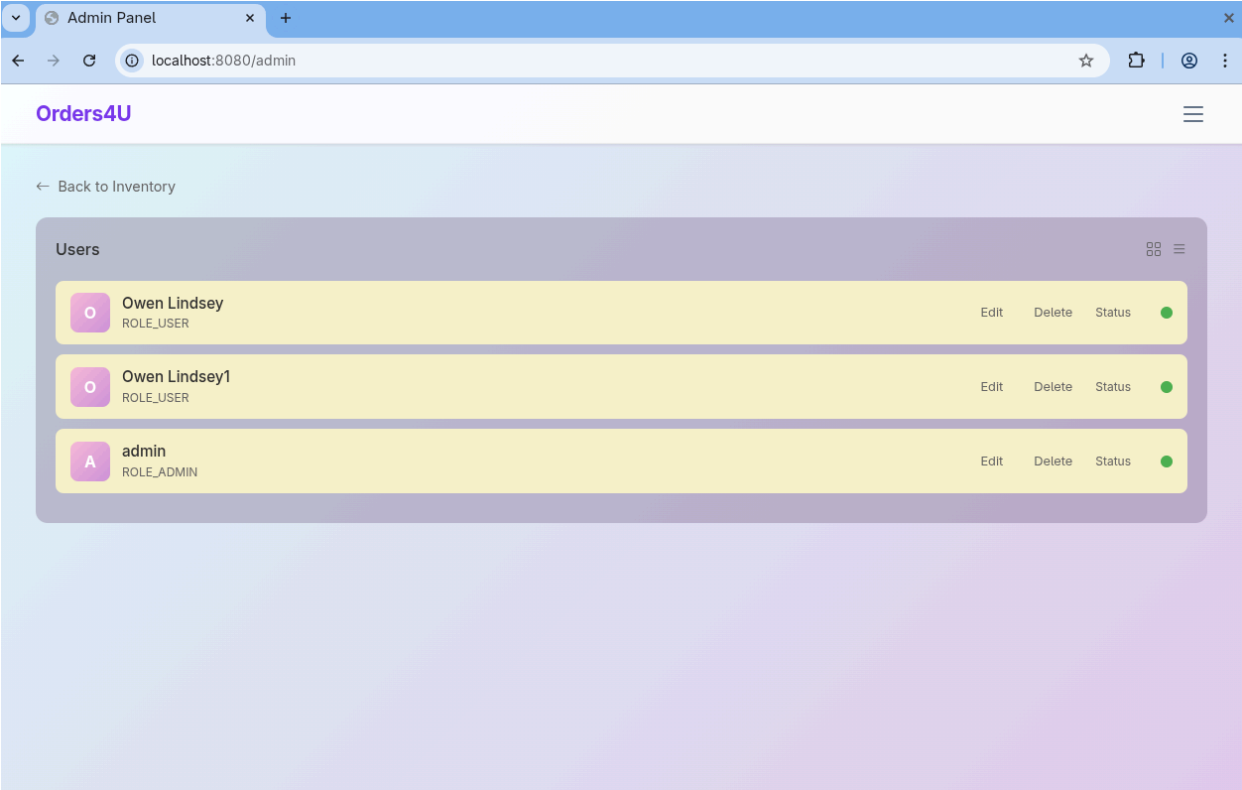
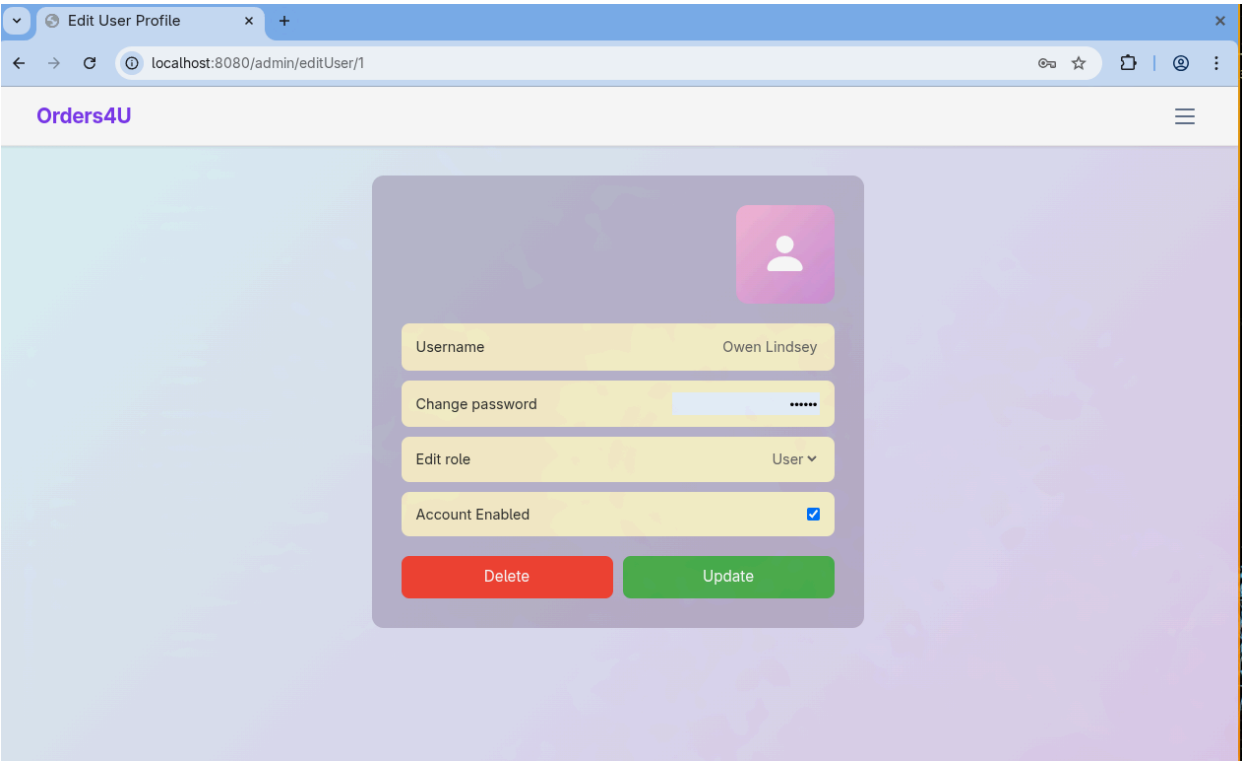
Authentication Pages

Page	Route	Screenshot
Login	/users/login	 A screenshot of a web browser displaying the login page. The page has a light purple gradient background. In the center, there is a white rounded rectangle containing the login form. The form has a title 'Login'. Below the title, there are two input fields: 'Username' with the value 'admin' and 'Password' with masked characters '*****'. Below the password field, there is a small text hint: 'at least 8 characters and one special character'. Below the inputs is a purple 'Login' button. At the bottom, there is a link: 'Don't have an account? Register'.
Register	/users/register	 A screenshot of a web browser displaying the register page. The page has a light purple gradient background. In the center, there is a white rounded rectangle containing the register form. The form has a title 'Register'. Below the title, there are three input fields: 'Username' with the value 'admin', 'Password' with masked characters '*****', and 'Confirm Password' with the placeholder text 'Confirm Password'. Below the password and confirm password fields, there are small text hints: 'at least 8 characters and one special character' for both. Below the inputs is a purple 'Register' button. At the bottom, there is a link: 'already have an account? login'.

Main Application Pages

Page	Route	Screenshot
Home	/	
Inventory (Orders)	/orders	

Admin Pages

Page	Route	Screenshot
Admin Panel	/admin	 A screenshot of a web browser showing the 'Admin Panel' at localhost:8080/admin. The page has a purple/pink gradient background and a header with 'Orders4U' and a hamburger menu. Below the header is a 'Back to Inventory' link. The main content area is titled 'Users' and contains a list of three users: 'Owen Lindsey' (ROLE_USER), 'Owen Lindsey1' (ROLE_USER), and 'admin' (ROLE_ADMIN). Each user entry has an avatar, name, role, and three action buttons: 'Edit', 'Delete', and 'Status' (with a green dot indicator).
Edit User	/admin/editUser/{id}	 A screenshot of a web browser showing the 'Edit User Profile' page at localhost:8080/admin/editUser/1. The page has a purple/pink gradient background and a header with 'Orders4U' and a hamburger menu. The main content area features a centered form for editing a user profile. The form includes a profile picture placeholder, fields for 'Username' (Owen Lindsey), 'Change password' (masked with dots), 'Edit role' (User), and 'Account Enabled' (checked). At the bottom of the form are two buttons: 'Delete' (red) and 'Update' (green).

UI Design

Element	Implementation
Color Scheme	Purple/pink gradient background
Layout	Card-based components
Navigation	"Orders4U" branding, hamburger menu
Admin Panel	User list with avatars, role badges, action buttons
Forms	Centered layout with validation hints